

EE102

Lab 2: Introduction to VHDL

Name	Zeynep Akçıl
Student ID	22102833
Section	2
Date	17.03.2023

Purpose

The aim of this lab is to design and implement a combinational circuit on BASYS 3 using VHDL. Combinational logic circuits are time-independent, memoryless circuits whose outputs solely depend on the combination of inputs. By doing this, an introduction to hardware designing, VHDL and programming with FPGA is targeted.

Design Specification

Multiplexers are components whose aim is to make selection according to selection signal. Depending on the number of inputted signals, the number of selection signals vary and these are allowing different types of multiplexers to be present for FPGA users. In this lab, a decomposed 6-to-1 multiplexer is designed. Because of the multi-usability of simpler components, it is practical to use smaller multiplexer blocks to create bigger ones. In this design, 5 2-to-1 multiplexer will be used. It will be 3-step-design which first selects 3 bits with the first and 1 bit with second and third selection signals and two of three outputs obtained will be used in step two, remaining signal obtained in step 1 and signal obtained step 2 will be used in step 3 to obtain mere signal. There will be 6 input signals in total (i1, i2, i3, i4, i5, i6), 3 selection signals (s1, s2, s3) and 1 output signal (out1) as expected from a multiplexer.

Input signals, i1, i2, i3, i4, i5 and i6, will be connected to R2, T1, U1, W2, R3 and T2 respectively. Selection signals, s1, s2 and s3, will be connected to W16, V16 and V17 respectively. Lastly, output signal, out1, will be displayed by connecting it to rightmost LED on the board which is U16.

Introduction

Before starting to the lab, here are some research questions:

1. How does one specify the inputs and outputs of a module in VHDL?

In the “entity” block, it is needed to open a “port” block limited with parenthesis. In port block, inputs and outputs are defined. To specify an input, it is needed to use the expression

```
inp : in std_logic;
```

and to specify an output, one needs to write

```
output1 : out std_logic;
```

Surely, the names written on the left hand side can be various; however, the right hand side must be exactly like this. (As a remark, at the very last row of the port block, “;” expression is not used. Instead, parenthesis is closed then semi colon is put. This can be found in the code attached to appendices section.)

After defining inputs and outputs, before generating bitstream, a constraint file is needed. The purpose of this is to tell compiler which switch or led or any other components on the board to use. Each component has a particular code (for instance, the code of first switch is “V17”) and by using these codes, these components are specified. In constraint file, common code is used. In this code file, designers uncomment rows according to what is needed. Others remain commented.

2. How does one use a module inside another code/module? What does PORT MAP do?

In an architecture block, port map maps inputs and outputs to ports.¹ By creating a port map and assigning local variables to other method’s variables, we become able to use another method out of the file currently worked on in a particular VHDL file.

In order to use another module in a file, it is necessary to use another entity. After defining inputs and outputs in main entity, architecture block should be located. Inside of this architecture block, after its “begin” code, another entity is needed. In this particular block, port map will be initialized.

These two features allow digital designers to use outer modules in VHDL files.

¹https://www.hdlworks.com/hdl_corner/vhdl_ref/VHDLContents/PortMap.htm#:~:text=A%20port%20map%20maps%20signals,association%20or%20via%20named%20association.

3. What is a constraint file? How does it relate your code to the pins on your FPGA?

The code that digital designers write, such as VHDL or Verilog codes, is digital. However, the switches or leds on BASYS 3 are analog. To communicate these components, we need to attribute what is and where is these analog components. By creating a constraint file, the physical ports which have two states, high or low, are addressed to the compiler. On the other hand, if testbench is used, there is no need to write a constraint file because in this case, no analog component works. Code is compiled by compiler and signals are generated and displayed in digital; no ports, no switches or leds, no location on a board but just variables and graphs consisting of their binary states.

4. What is the purpose of writing a testbench?

When physical board, for instance, BASYS 3, is not available, testing the code and progressing on work might be problematic. If this is the case, testbenches can be very useful. Or in business, projects are big and it takes a noteworthy time to implement the code. In these cases instead of connecting the board and trying to code each time, engineers can write a simple testbench and according to results, they can move on. For example, if there are problems in the code, if it is not working in desired way, they can easily detect the problems and fix them. Therefore, testbench, as a practical simulation mechanism, is really helpful way to choose for digital designers.

Methodology

Steps will be followed are: designing the circuit on the paper, writing its code, displaying elaborated design, creating simulation, writing constraints and lastly, embedding written VHDL code to BASYS 3 by generating bitstream.

In VHDL code, as an introductory block, there is “entity” part. In this block, inputs and outputs (and inouts if needed) are declared. Then, there is an “architecture” block. This part is the main part which holds the behavior of design. In first line of the architecture clock, signals are defined. Signals are likely variables that carry data from one place to another and they are useful because they can be obtained as output in one step and can be used as an input in the next step. After that, to write the main code, a process block is used in which there are a couple of if-else mechanisms are used. They are logically designed depending on the multiplexer design specification which selects first input if selection signal is 0, second input if selection signal is 1. After writing whole if-else blocks, process and architecture blocks are ended and design code is done. This code can be found in appendix.

In order to visualize the code without board, test bench is coded. At first, in order to define them, all unknowns are set to 0. Period times are given arbitrarily and according to these time periods, results are different. In UUT, there is a port map which connects the variables in main code and in test bench. After port map, process blocks are present in which changes in values are made.

Additionally, a constraint file is written. This is an universal code which is not written by me. This file marks the ports and variable connected to that particular port. I copied constraint file present on Moodle and on the internet, uncommented ports used.

Results

1. After the implementation of the given circuit and testbench, this output waveform and RTL design below is obtained.

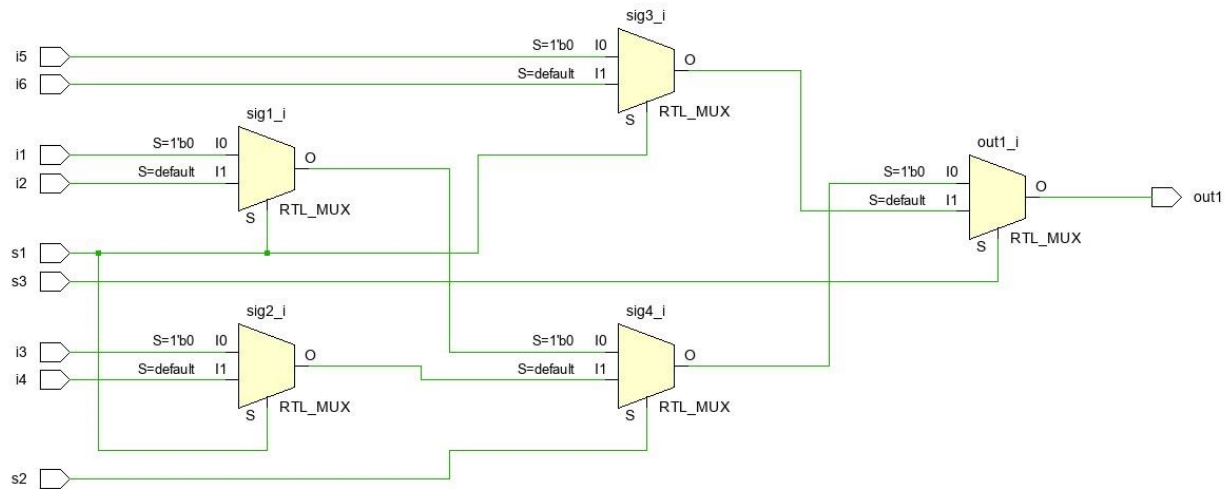


Figure 1: RTL design

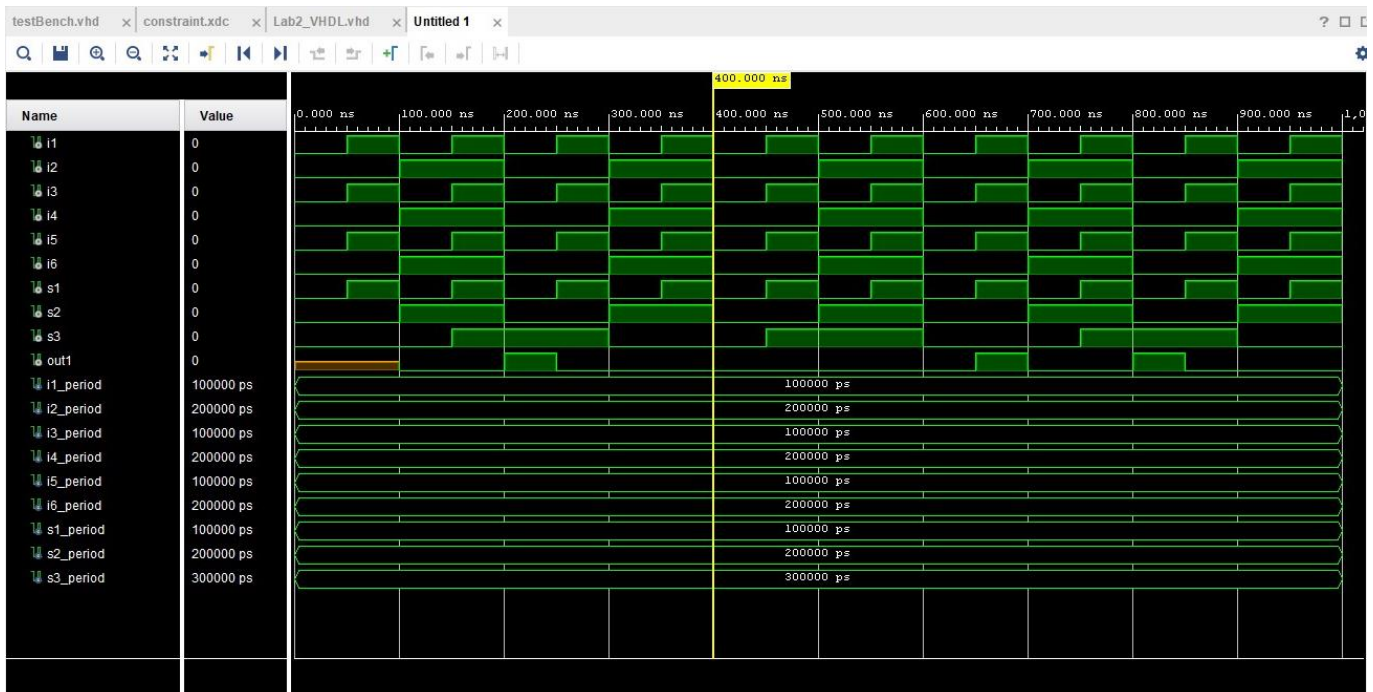


Figure 2: Output waveform

2. After programming board, as it is seen 8-to-1 multiplexer is present. In the photo below, first input, i_1 is got. When it is 1, LED is on and when it is 0, LED is off. In the appendix, more photos can be found.

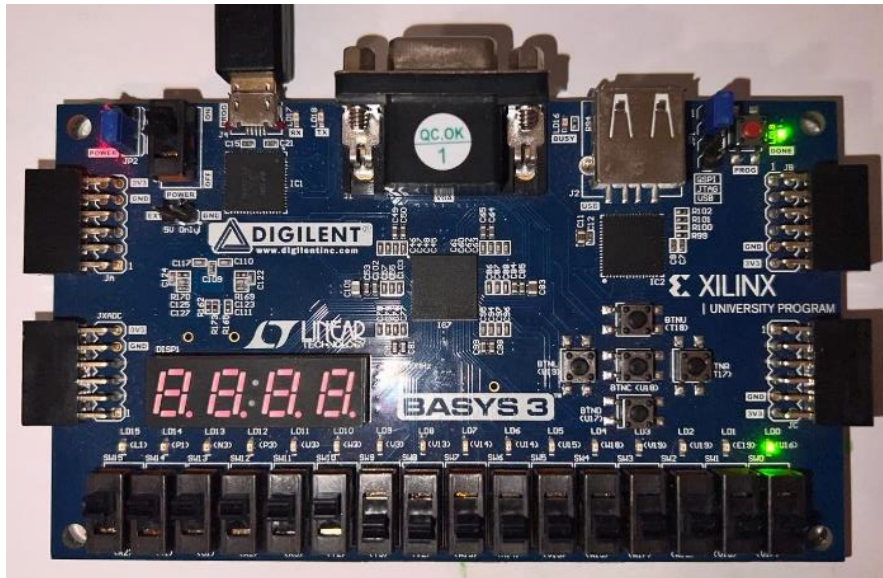


Figure 3: $s_1=0$, $s_2=0$, $s_3=0$, $i_1=1$. LED is ON.

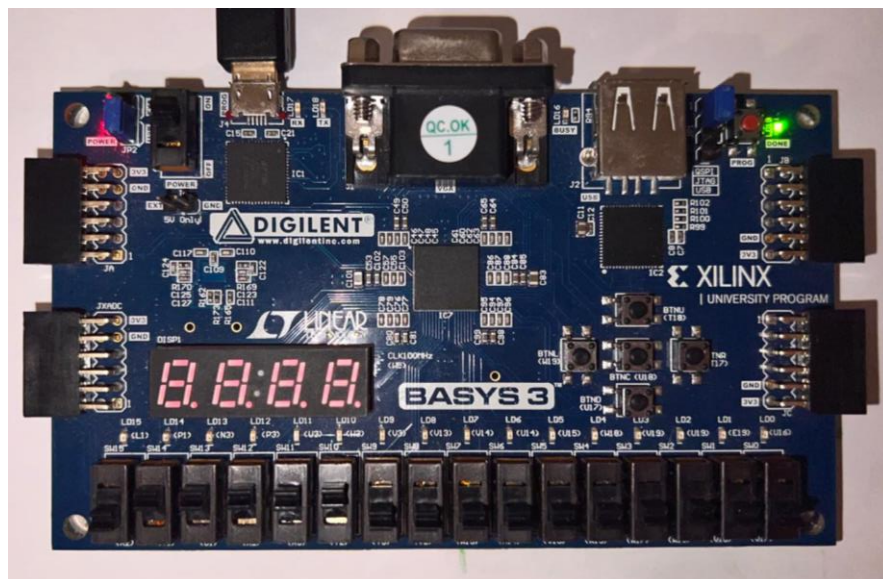


Figure 4: $s_1=0$, $s_2=0$, $s_3=0$, $i_1=0$. LED is OFF.

Conclusion

The aim of this experiment was to introduce VHDL with basic components such as main VHDL code, test bench, constraint file, waveform graph and schematics. In this particular experiment, a combinational circuit is designed. The circuit is 6-to-1 multiplexer which allows digital designers to select mere data among 6 input signals with 3 selection signals. Also since this design is composed of 5 2-to-1 multiplexers, reusability is taken into account. As the photos in the appendices, multiplexer works as expected.

This circuit can be used in CD players. For example, a radio player having 6-to-1 multiplexer from it can select particular recording using selection inputs inputted through buttons on the player. When a button is pushed, namely ON, a binary value of '1' is transferred to the multiplexer inside the player and the circuit designed in this lab works to select only one audio.

Appendices

1. Code of the design source:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Lab2_VHDL is
  Port (i1  : in std_logic;
        i2  : in std_logic;
        i3  : in std_logic;
        i4  : in std_logic;
```

```

        i5  : in std_logic;
        i6  : in std_logic;
        s1  : in std_logic;
        s2  : in std_logic;
        s3  : in std_logic;
        out1: out std_logic);
end Lab2_VHDL;

architecture Behavioral of Lab2_VHDL is
    signal sig1, sig2, sig3, sig4 : std_logic;
begin
    process(i1,i2,i3,i4,i5,i6,s1,s2)
    begin
        if s1 = '0' then
            sig1 <= i1;
            sig2 <= i3;
            sig3 <= i5;
            if s2 = '0' then
                sig4 <= sig1;
                if s3 = '0' then
                    out1 <= sig4;
                else
                    out1 <= sig3;
                end if;
            else
                sig4 <= sig2;
                if s3 = '0' then
                    out1 <= sig4;
                else
                    out1 <= sig3;
                end if;
            end if;
        end if;
    end process;
end Behavioral;

```



```

        end if;
    end if;
else
    sig1 <= i2;
    sig2 <= i4;
    sig3 <= i6;
    if s2 = '0' then
        sig4 <= sig1;
        if s3 = '0' then
            out1 <= sig4;
        else
            out1 <= sig3;
        end if;
    else
        sig4 <= sig2;
        if s3 = '0' then
            out1 <= sig4;
        else
            out1 <= sig3;
        end if;
    end if;
end if;

end process;

end Behavioral;

```

2. Code of the testbench:

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values

```

```

--use IEEE.NUMERIC_STD.ALL;
-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity testBench is
-- Port ( );
end testBench;

architecture Behavioral of testBench is

component Lab2_VHDL PORT(
    i1 : in std_logic;
    i2 : in std_logic;
    i3 : in std_logic;
    i4 : in std_logic;
    i5 : in std_logic;
    i6 : in std_logic;

    s1 : in std_logic;
    s2 : in std_logic;
    s3 : in std_logic;

    out1 : out std_logic);
end component;

signal i1 : std_logic := '0';
signal i2 : std_logic := '0';
signal i3 : std_logic := '0';

```

```

signal i4 : std_logic := '0';
signal i5 : std_logic := '0';
signal i6 : std_logic := '0';

signal s1 : std_logic := '0';
signal s2 : std_logic := '0';
signal s3 : std_logic := '0';

signal out1:  std_logic;

-- Constant Times of Inputs
constant i1_period : time := 100ns;
constant i2_period : time := 200ns;
constant i3_period : time := 100ns;
constant i4_period : time := 200ns;
constant i5_period : time := 100ns;
constant i6_period : time := 200ns;

constant s1_period : time := 100ns;
constant s2_period : time := 200ns;
constant s3_period : time := 300ns;

begin

-- UUT: Unit Under Test
UUT: Lab2_VHDL PORT MAP(
    i1 => i1,
    i2 => i2,
    i3 => i3,
    i4 => i4,
    i5 => i5,

```

```
i6 => i6,
```

```
s1 => s1,
```

```
s2 => s2,
```

```
s3 => s3,
```

```
out1 => out1);
```

```
P_i1: process begin
```

```
    i1 <= '0';
```

```
    wait for i1_period/2;
```

```
    i1 <= '1';
```

```
    wait for i1_period/2;
```

```
end process;
```

```
P_i2: process begin
```

```
    i2 <= '0';
```

```
    wait for i2_period/2;
```

```
    i2 <= '1';
```

```
    wait for i2_period/2;
```

```
end process;
```

```
P_i3: process begin
```

```
    i3 <= '0';
```

```
    wait for i3_period/2;
```

```
    i3 <= '1';
```

```
    wait for i3_period/2;
```

```
end process;
```

```
P_i4: process begin
```

```
    i4 <= '0';
```

```
    wait for i4_period/2;
```

```
    i4 <= '1';
```

```

        wait for i4_period/2;
end process;
P_i5: process begin
    i5 <= '0';
    wait for i5_period/2;
    i5 <= '1';
    wait for i5_period/2;
end process;
P_i6: process begin
    i6 <= '0';
    wait for i6_period/2;
    i6 <= '1';
    wait for i6_period/2;
end process;
P_s1: process begin
    s1 <= '0';
    wait for s1_period/2;
    s1 <= '1';
    wait for s1_period/2;
end process;
P_s2: process begin
    s2 <= '0';
    wait for s2_period/2;
    s2 <= '1';
    wait for s2_period/2;
end process;
P_s3: process begin
    s3 <= '0';
    wait for s3_period/2;
    s3 <= '1';

```

```

        wait for s3_period/2;
    end process;

MUX8to1: process begin

    wait for 100 ms; -- wait 100 ms at first

    wait for i1_period*10; -- give a break for 10 i1 per.long

    wait;

end process;

end;
```

3. Truth Table

Since 8-to-1 multiplexer has 11 inputs in total, its truth table is highly long and hard to be displayed in here. Therefore, with the approval of TA, truth table of 2-to-1 multiplexer, which is the component of 8-to-1, is shown. Also in the second table, a couple of lines of truth table of 8-to-1 multiplexer is written to exemplify.

i1	i2	s	output
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Table 1: Truth Table of 2-to-1 multiplexer

i1	i2	i3	i4	i5	i6	s1	s2	s3	out1
0	0	0	0	1	1	0	0	0	0
0	0	1	1	1	1	0	0	1	1
0	1	1	1	1	0	1	1	0	1
1	0	0	0	0	1	1	1	1	1
1	0	1	1	0	1	1	1	0	1

Table 2: 5 random lines from truth table of 8-to-1 multiplexer

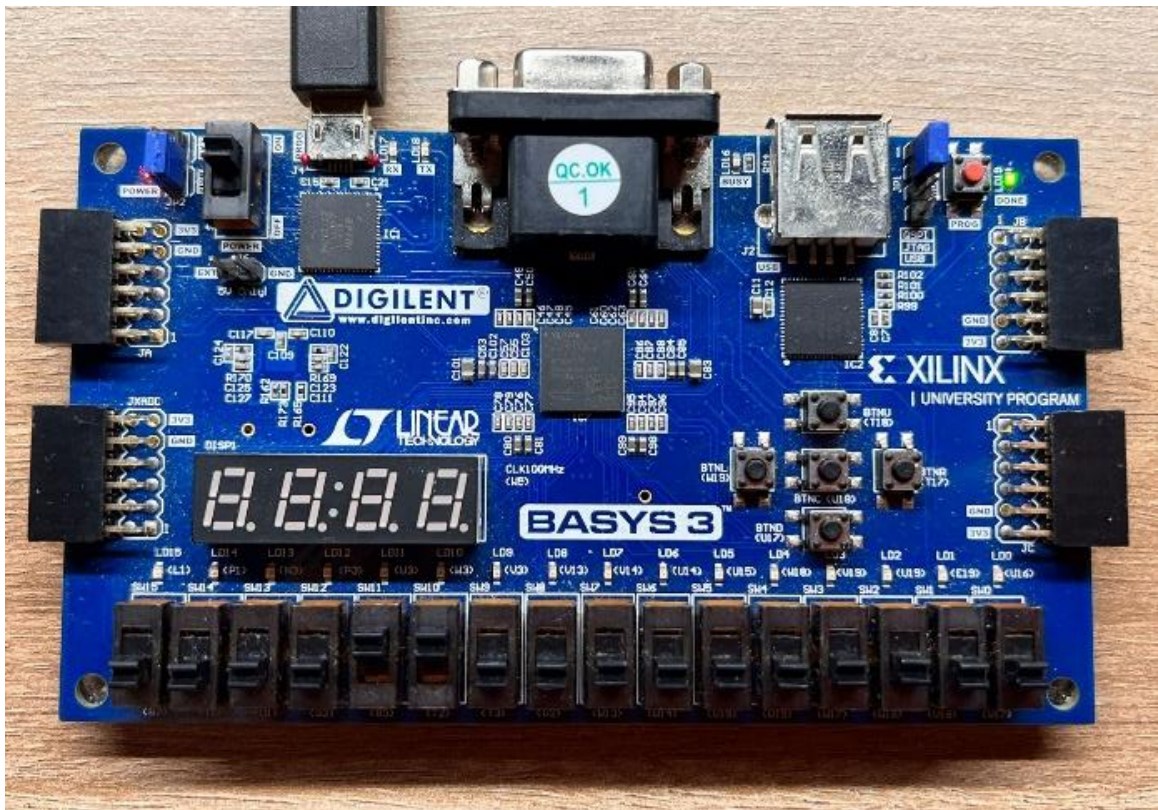


Figure 5: s1, s2 and s3 is off.

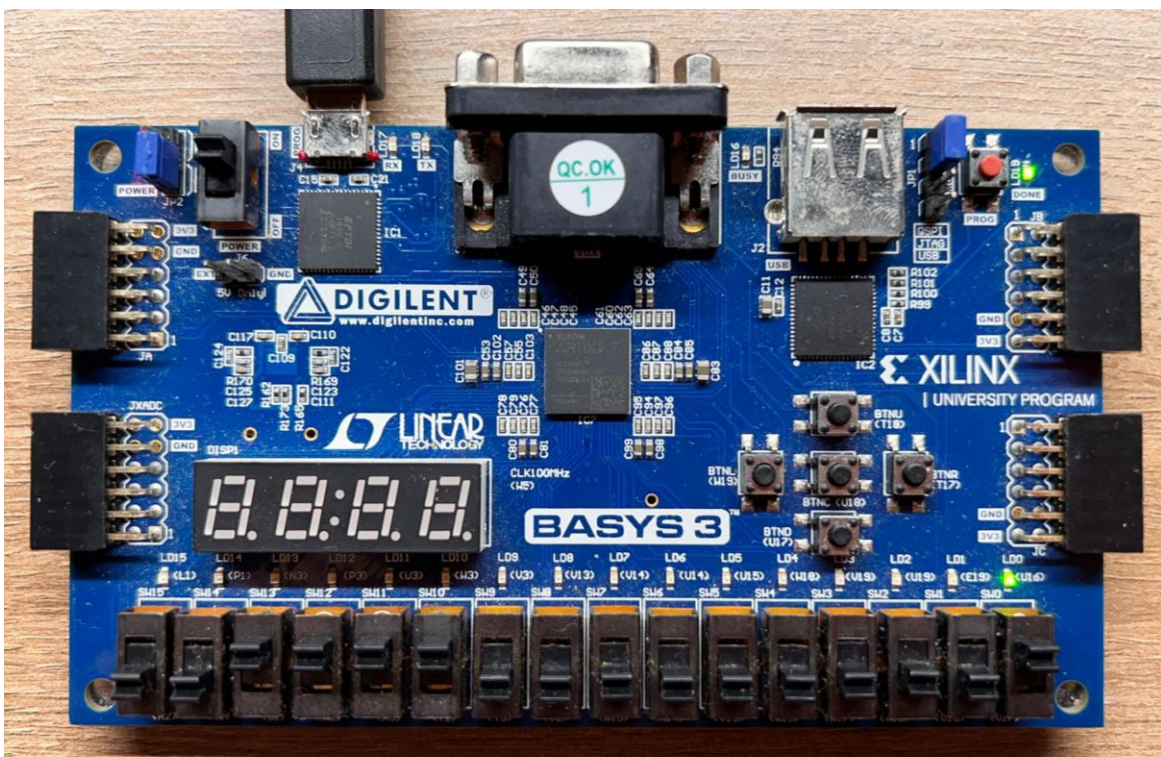


Figure 6: s1 and s2 is off, s3 is on.

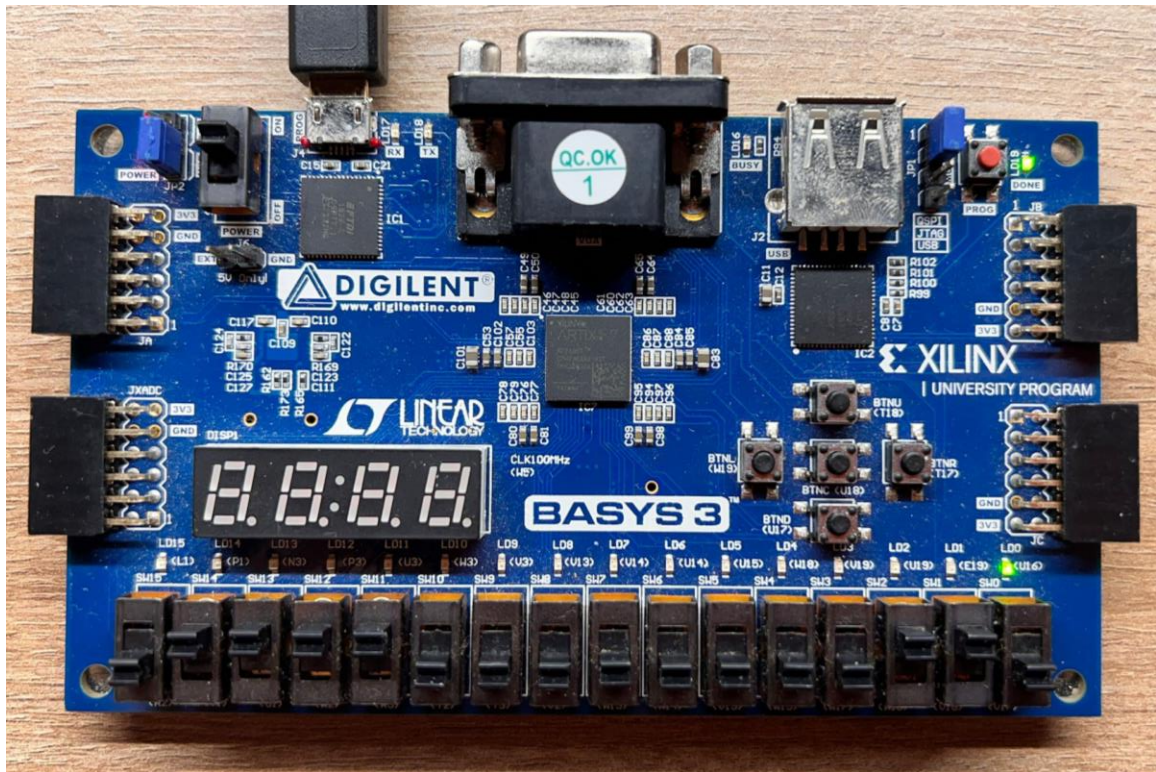


Figure 7: s1 and s2 is on, s3 is off.

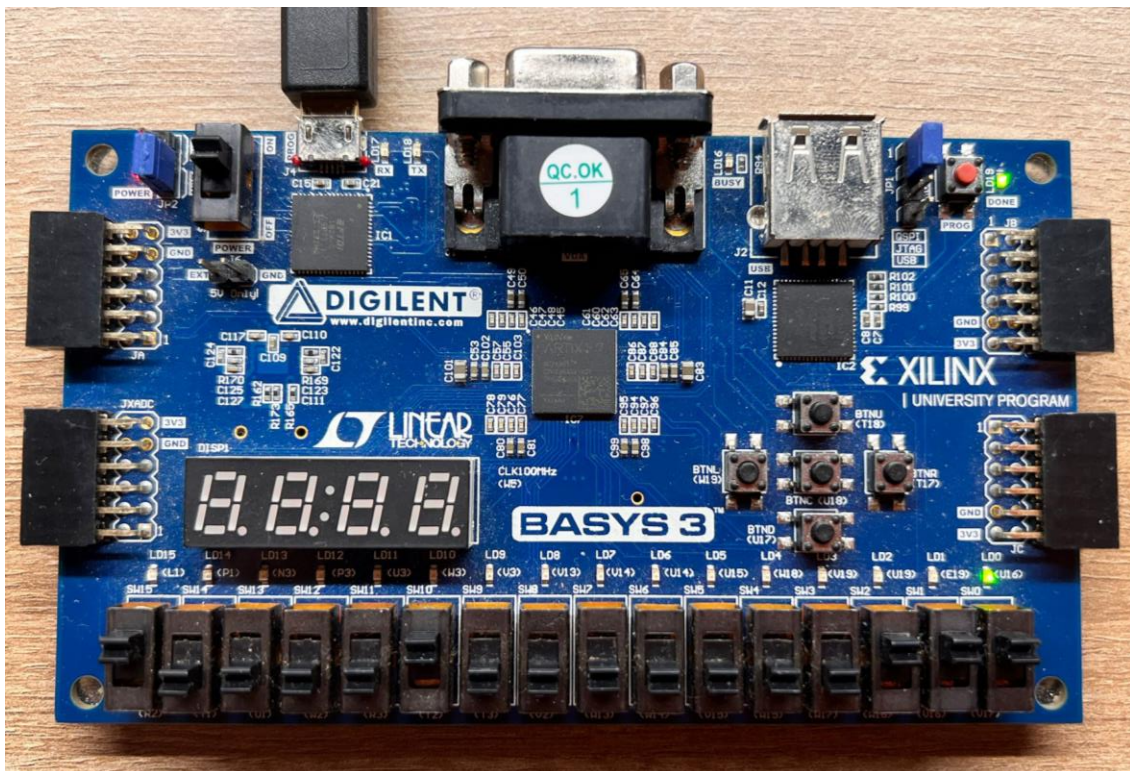


Figure 8: s1, s2 and s3 is on.

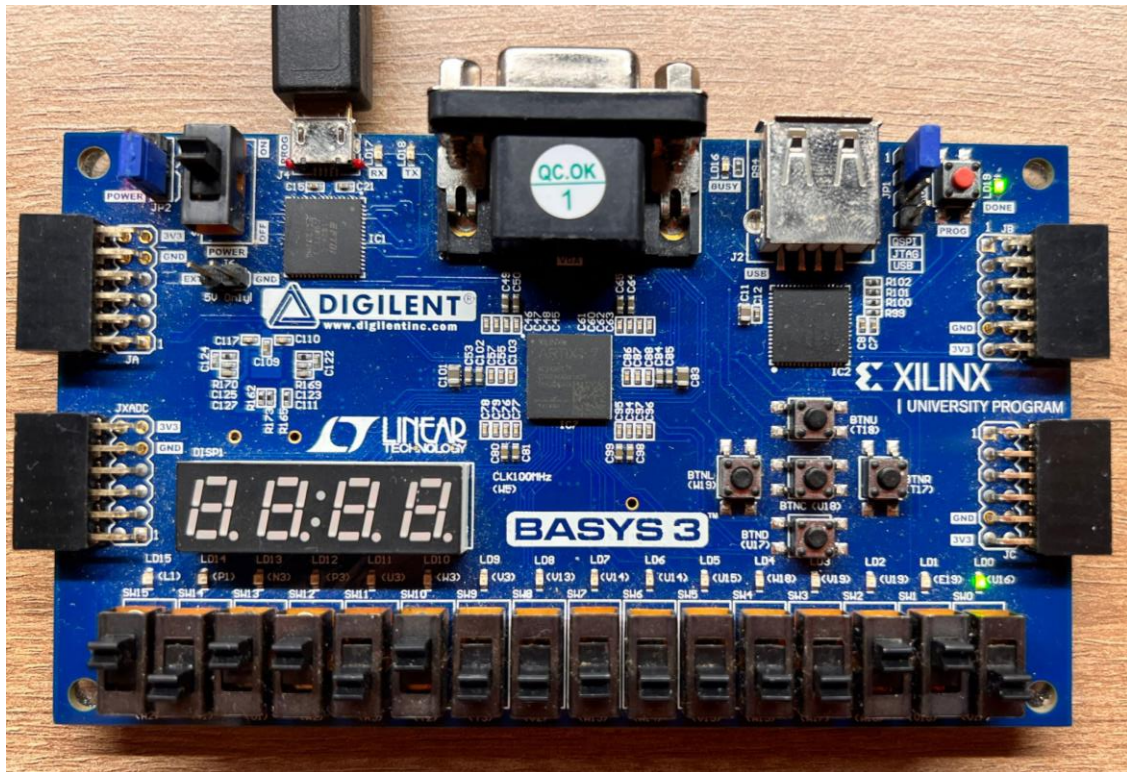


Figure 9: s1 and s2 is on, s3 is off.

References

https://www.hdlworks.com/hdl_corner/vhdl_ref/VHDLContents/PortMap.htm#:~:text=A%20port%20map%20maps%20signals,association%20or%20via%20named%20association.